engineering proceedings

MDPI

Conference Report

Algorithms for Finding the Shortest Path in a Graph: A Comparative Implementation in the Game *PetWars* †

Hristo Nikolaev Ivanov [1,*]

[1] Department of Computer Sciences, Faculty of Social, Economic and Computer Sciences, Varna Free University *"Chernorizets Hrabar"*, 84 Yanko Slavchev Str., Chaika Resort, 9007 Varna, Bulgaria; *257330006@vfu.bg*

* Correspondence: *257330006@vfu.bg*

† Presented as a course project for the discipline *Software Systems Design* (*Проектиране на софтуерни системи*), Master's programme, Varna Free University, Varna, Bulgaria, 2026.

Abstract

This study presents a practical implementation and empirical comparison of three classical shortest-path algorithms-Dijkstra's algorithm, the A* algorithm, and the Bellman-Ford algorithm-within the context of a turn-based strategy game, PetWars. The game models a 14×10 grid of walkable and blocked tiles, in which a hero moves in eight directions (4 orthogonal, 4 diagonal) using a weighted cost of 2 for orthogonal moves and 3 for diagonal moves. A dedicated demo mode visualises each algorithm step by step, exposing the frontier set, the visited set and the resulting shortest path. The three algorithms are benchmarked across short, medium and long target distances; for each configuration, execution time and the number of expanded vertices are measured. The experimental results show that A* is the fastest algorithm across all distances thanks to its Octile-distance heuristic, while Dijkstra explores significantly more vertices as distance grows, and Bellman–Ford remains the slowest with a near-constant exploration cost due to its O(V · E) edge-relaxation procedure. All three algorithms recover paths of identical length, confirming the correctness of the implementation. The work demonstrates that the choice of algorithm in grid-based games has a measurable impact on responsiveness even on small maps, and provides a reusable Python reference for educational use.

Keywords: shortest path algorithms; Dijkstra; A*; Bellman-Ford; pathfinding; grid game; octile heuristic; Python; Pygame; PetWars

Academic Editor: Sapundzhi, F.

Received: 2026
Accepted: 2026
Published: 2026

Citation: Ivanov, H. N. Algorithms for Finding the Shortest Path in a Graph: A Comparative Implementation in the Game PetWars. *VFU Course Proc.* 2026, 1, 1.

1. Introduction

Shortest-path algorithms are one of the most fundamental concepts in graph theory and computer science. They have applications across many domains, from GPS navigation and network protocols to artificial intelligence and computer games [1–3]. Even small grid-based games rely on efficient pathfinding for both player input (clicking a tile to move) and AI behaviour, where responsiveness directly affects perceived quality.

This course project presents a practical implementation of three classical shortest-path algorithms-Dijkstra's algorithm, the A* algorithm, and the Bellman-Ford algorithm-and an empirical comparison of their performance in the game PetWars, a turn-based strategy game inspired by the *Heroes of Might and Magic* series. In PetWars, the algorithms are used to compute the optimal path from the hero's current position to a tile selected by

mouse click. A demo mode developed specifically for this project visualises the work of each algorithm step by step, making the project also suitable for educational purposes.

The remainder of this paper is organised as follows. Section 2 reviews the theoretical background of the three algorithms and presents a worked example of Dijkstra's algorithm on a small graph. Section 3 describes the software architecture and the Python implementations. Section 4 details the demonstration and demo-mode visualisation. Section 5 reports the experimental setup and benchmark results at short, medium and long distances. Section 6 discusses the results, and Section 7 concludes with directions for future work.

2. Theoretical Background

2.1. The Selected Algorithms

Three shortest-path algorithms are implemented and compared in this work:

- Dijkstra's algorithm - a classical algorithm for graphs with non-negative edge weights.
- A* algorithm - an extension of Dijkstra with an admissible heuristic function.
- Bellman-Ford algorithm - a general-purpose algorithm that also works with negative edge weights.

These algorithms are fundamental in graph theory and are widely used in computer games, navigation systems and network protocols [4,5].

2.2. Dijkstra's Algorithm

Dijkstra's algorithm is a classical method for finding the shortest path from a single source to all other vertices in a graph with non-negative edge weights. It was developed by Edsger W. Dijkstra in 1956 [5]. The algorithm maintains a set of visited vertices and a priority queue of unvisited vertices. For each vertex it stores the current shortest distance from the source, and at every step the unvisited vertex with the smallest distance is selected and its neighbours are relaxed.

Formally, let G = (V, E) be a graph with vertex set V and edge set E. For every edge $(u, v) \in E$ there is a weight $w(u, v) \geq 0$. The algorithm finds $d[v]$ - the minimum distance from the starting vertex s to every vertex $v \in V$. Typical applications include GPS navigation and routing, link-state network protocols such as OSPF, and AI pathfinding in computer games. The classical implementation using a binary heap runs in O((V + E) log V) time.

2.3. A Algorithm*

The A* algorithm is an extension of Dijkstra's algorithm that uses a heuristic function to guide the search towards the goal, which makes it significantly more efficient in practical applications. A* uses the evaluation function f(n) = g(n) + h(n), where g(n) is the actual cost from the source to the current vertex and h(n) is the heuristic estimate from the current vertex to the goal.

In this project the *octile distance* is used as the heuristic, since it is admissible for movement in 8 directions (including diagonal): $h(n) = D \cdot \max(|dx|, |dy|) + (D_2 - D) \cdot \min(|dx|, |dy|)$, where D = 2 (orthogonal cost) and $D_2 = 3$ (diagonal cost). With these constants the heuristic simplifies to $h(n) = 2 \cdot \max(|dx|, |dy|) + \min(|dx|, |dy|)$.

The heuristic h(n) must be admissible - i.e. it must never overestimate the real distance to the goal - in order for A* to remain optimal. The octile distance satisfies this property for 4-orthogonal + 4-diagonal movement. Typical applications include AI

pathfinding in computer games, robotics and autonomous vehicles, and task planning. The worst-case complexity is O((V + E) log V), but in practice A* is much faster than Dijkstra.

2.4. Bellman-Ford Algorithm

The Bellman-Ford algorithm finds the shortest paths from a single source to all vertices and - unlike Dijkstra - can also work with negative edge weights. It performs V - 1 iterations and at each iteration relaxes all edges in the graph: for every edge (u, v) with weight w(u, v), if d[u] + w(u, v) < d[v] then d[v] is updated to d[u] + w(u, v). After V - 1 iterations, if there are still edges that can be relaxed, the graph contains a negative cycle. Typical applications include distance-vector network protocols (RIP, BGP), negative-cycle detection and financial arbitrage. The complexity is O(V · E), slower than Dijkstra in dense settings but more general.

2.5. Comparison of the Algorithms

Table 1. Comparison of the three implemented algorithms.

Algorithm	Complexity	Advantages	Disadvantages
Dijkstra	O((V + E) log V)	Optimal, reliable	Explores in all directions
A*	O((V + E) log V)	Guided search, fast	Requires a good heuristic
Bellman-Ford	O(V · E)	Works with negative weights	Slower in practice

All three algorithms use the same 8-direction movement model with a cost of 2 for orthogonal moves and 3 for diagonal moves.

2.6. Worked Example: Dijkstra on a Small Graph

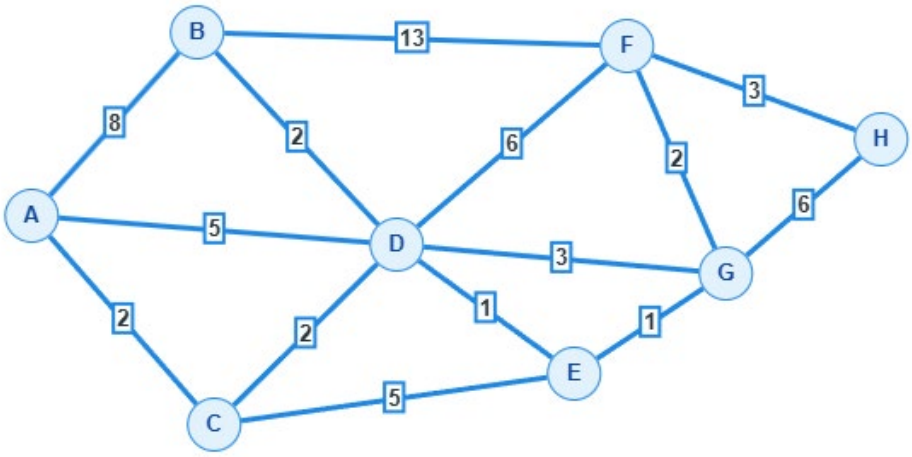

Figure 1. Example weighted undirected graph used to illustrate Dijkstra's algorithm.

Table 2. Step-by-step relaxation when running Dijkstra's algorithm from A to H.

V	A	B	C	D	E	F	G	H
A	0^{A}	8^{A}	2^{A}	5^{A}	∞	∞	∞	∞
C		8^{A}	2^{A}	4^{c}	7^{c}	∞	∞	∞
D		6^{D}		4^{c}	5^{D}	10^{D}	7^{D}	∞
E		6^{D}			5^{D}	10^{D}	6^{E}	∞
B		6^{D}				10^{D}	6^{E}	∞
G						8^{g}	6^{E}	12^{g}
F						8^{g}		11^{f}
H								11^{f}

Dijkstra's algorithm proceeds as follows. The source vertex A is initialised with distance 0 and all other vertices with infinity. At each iteration, the unvisited vertex with the smallest current distance is selected and "locked"-its distance is final. The distances of all of its neighbours are then updated whenever a shorter path can be obtained through the locked vertex. The procedure terminates when the target vertex is locked or no further relaxation is possible. In the table above, the subscript next to each value records through which predecessor the current shortest path passes.

Iteration 1. Visit A. Distance to A = 0. Neighbours: B = 0 + 8 = 8^{A}, C = 0 + 2 = 2^{A}, D = 0 + 5 = 5^{A}; the remaining vertices are at infinity. Lock A = 0. Next: C.

Iteration 2. Visit C. Distance to C = 2. Updates: D = min(5, 2 + 2) = 4^{c}; E = 2 + 5 = 7^{c}. Lock C = 2. Next: D.

Iteration 3. Visit D. Distance to D = 4. Updates: B = min(8, 4 + 2) = 6^{D}; E = min(7, 4 + 1) = 5^{D}; F = 4 + 6 = 10^{D}; G = 4 + 3 = 7^{D}. Lock D = 4. Next: E.

Iteration 4. Visit E. Distance to E = 5. Updates: G = min(7, 5 + 1) = 6^{E}. Lock E = 5. Next: B.

Iteration 5. Visit B. Distance to B = 6. F: min(10, 6 + 13) = 10^{D} (unchanged). Lock B = 6. Next: G.

Iteration 6. Visit G. Distance to G = 6. Updates: F = min(10, 6 + 2) = 8^{g}; H = 6 + 6 = 12^{g}. Lock G = 6. Next: F.

Iteration 7. Visit F. Distance to F = 8. Updates: H = min(12, 8 + 3) = 11^{f}. Lock F = 8. Next: H.

Iteration 8. Visit H. Distance to H = 11. No unvisited neighbours remain. Lock H = 11. The algorithm terminates.

Result: the shortest path from A to H has total cost 11, with path A → C → D → E → G → F → H.

3. Software Implementation

The project is implemented in Python using the *Pygame* library for the graphical interface. The full source code is publicly available at *https://github.com/hristogwivanov/PetWars*. The main modules are listed below.

- main.py - main file with the game loop and rendering.
- pathfinding.py - implementation of the three algorithms.
- gamedata.py - classes for heroes, resources and the AI opponent.
- constants.py - constants, colours and terrain maps.
- interface.py - UI components (buttons, turn counter).

3.1. Dijkstra Implementation

```
function DIJKSTRA(start, goal, grid):
    dist[start] ← 0                      # every other distance ← ∞
    prev ← empty map                      # predecessor of each vertex
    Q ← priority queue holding (0, start)

    while Q is not empty:
        (d, u) ← remove vertex with smallest distance from Q
        if u = goal:
            return path rebuilt from prev
        if d > dist[u]:
            continue                      # outdated queue entry, skip
        for each neighbour v of u (4 orthogonal + 4 diagonal):
            cost ← 2 if the move is orthogonal else 3
            if dist[u] + cost < dist[v]:
                dist[v] ← dist[u] + cost
                prev[v]  ← u
                insert (dist[v], v) into Q

    return "no path found"
```

Listing 1. Pseudocode of Dijkstra's algorithm in PetWars.

3.2. A Implementation with Octile Heuristic*

```
function OCTILE(a, b):                      # admissible heuristic (D = 2, D₂ = 3)
    dx ← |a.x − b.x|;   dy ← |a.y − b.y|
    return 2 · max(dx, dy) + 1 · min(dx, dy)

function A*(start, goal, grid):
    g[start] ← 0                          # cost from start; others ← ∞
    prev ← empty map
    Q ← priority queue holding (OCTILE(start, goal), start)
    visited ← empty set

    while Q is not empty:
        u ← vertex in Q with smallest f = g + h
        if u = goal:
            return path rebuilt from prev
        if u in visited:
            continue
        add u to visited
        for each neighbour v of u (4 orthogonal + 4 diagonal):
            if v in visited:
                continue
            cost ← 2 if the move is orthogonal else 3
            if g[u] + cost < g[v]:
                g[v]   ← g[u] + cost
                prev[v] ← u
                f ← g[v] + OCTILE(v, goal)
                insert (f, v) into Q

    return "no path found"
```

Listing 2. Pseudocode of A* with the octile heuristic.

3.3. Bellman-Ford Implementation

```
function BELLMAN-FORD(start, goal, grid):
    V ← all walkable cells
    E ← all edges (u, v, cost) between adjacent walkable cells
                                          # cost = 2 orthogonal, 3 diagonal
    for each vertex v in V:
        dist[v] ← ∞
    dist[start] ← 0
    prev ← empty map

    repeat |V| − 1 times:
        updated ← false
        for each edge (u, v, cost) in E:
            if dist[u] + cost < dist[v]:
                dist[v] ← dist[u] + cost
                prev[v] ← u
                updated ← true
        if not updated:
            break                         # converged, stop early

    if dist[goal] = ∞:
        return "no path found"
    return path rebuilt from prev
```

Listing 3. Pseudocode of Bellman-Ford.

4. Demonstration

Figure 2. Start screen of the game PetWars.

The map is represented as a two-dimensional grid of 14×10 cells, where each cell is either walkable (value 1) or blocked (value 0). The hero starts in the bottom-left corner at coordinates (1, 8). A target cell is selected with a mouse click, and the algorithms are applied as follows:

1. Initialisation: the starting position receives distance 0, all other vertices ∞.
2. The starting position is added to the priority queue.
3. The vertex with the smallest current distance is extracted from the queue.
4. Each of its neighbours is examined (4 orthogonal + 4 diagonal moves, with costs 2 and 3 respectively).
5. Steps 3–4 repeat until the goal is reached or the queue is empty.
6. The path is reconstructed by backtracking from the goal to the start.

In demo mode, the visualisation distinguishes four kinds of cells: orange cells for already-visited vertices, yellow cells for vertices currently in the queue (frontier), a red cell for the vertex currently being expanded, and blue cells for the final shortest path.

Figure 3. Visualisation of the algorithm in demo mode (prior to the introduction of diagonal movement).

Demo mode is activated with the DEMO MODE button or the D key. The visualisation runs step by step with an adjustable speed (200 ms between steps), and the algorithm under inspection is selected with the Algo button. The principal controls are: arrow keys to move the hero in 4 directions, mouse click to compute and follow a path to the selected tile, D to toggle demo mode, and E to end the turn.

During the project, three features were added to the original engine: a movement-points system (the hero has 10 movement points per turn, with orthogonal cost 2 and diagonal cost 3), a smooth path rendering using Chaikin's corner-cutting algorithm that produces arcs instead of sharp turns, and full 8-direction movement.

5. Experimental Setup and Results

To compare the three algorithms, benchmarks were performed at three different distances (short, medium and long). For each test the execution time and the number of expanded vertices were measured. Each configuration was repeated three times.

5.1. Short Distance

Figure 4. Dijkstra's algorithm at a short distance.

Starting position: (9, 2) → target: (7, 4).

Table 3. Benchmark results at the short distance.

Algorithm	Time (ms)	Visited vertices	Path length
Dijkstra 1	0.068	10	10
Dijkstra 2	0.090	10	10
Dijkstra 3	0.073	10	10
A* 1	0.088	6	10
A* 2	0.077	6	10
A* 3	0.062	6	10
Bellman-Ford 1	0.617	59	10
Bellman-Ford 2	0.678	59	10
Bellman-Ford 3	0.792	59	10

5.2. Medium Distance

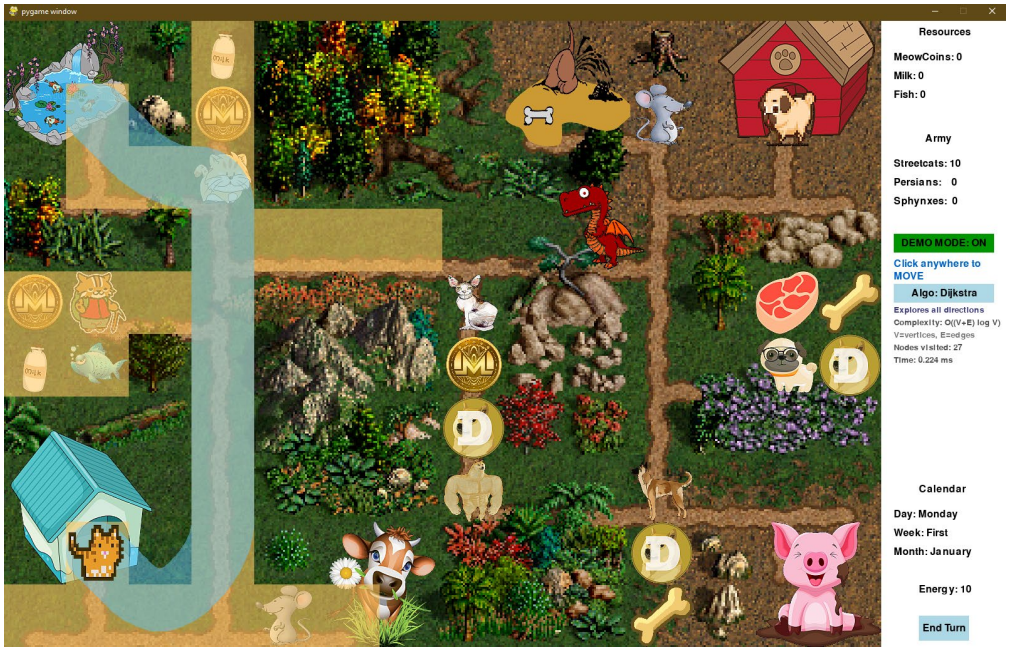

Figure 5. Dijkstra's algorithm at a medium distance.

Starting position: (9, 2) → target: (2, 2).

Table 4. Benchmark results at the medium distance.

Algorithm	Time (ms)	Visited vertices	Path length
Dijkstra 1	0.224	27	22
Dijkstra 2	0.185	27	22
Dijkstra 3	0.203	27	22
A* 1	0.161	16	22
A* 2	0.135	16	22
A* 3	0.131	16	22
Bellman-Ford 1	0.559	59	22
Bellman-Ford 2	0.564	59	22
Bellman-Ford 3	0.832	59	22

5.3. Long Distance

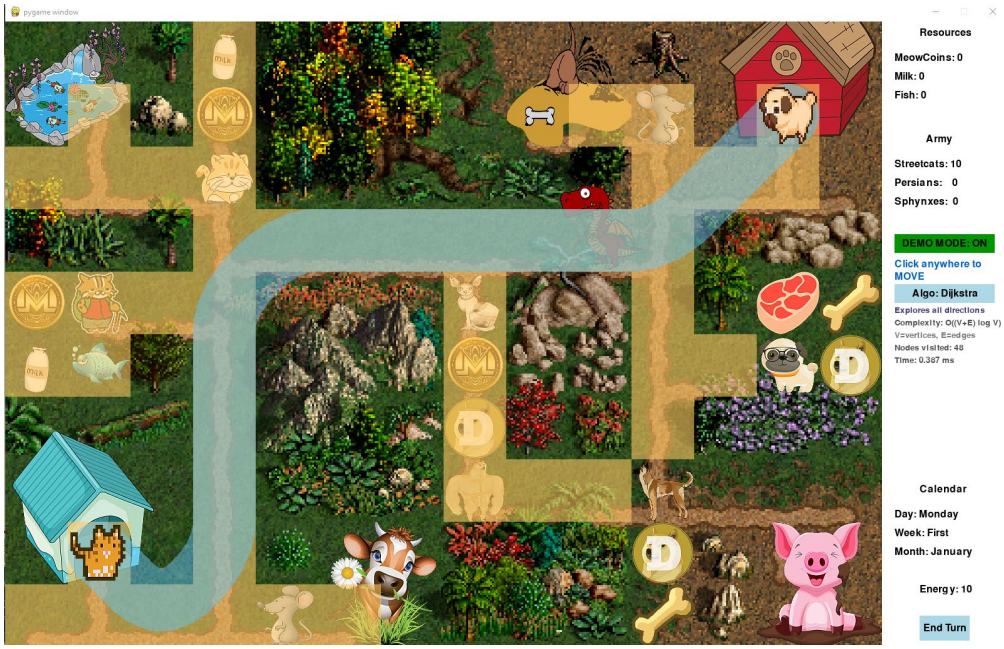

Figure 6. Dijkstra's algorithm at a long distance.

Starting position: (9, 2) → target: (2, 13).

Table 5. Benchmark results at the long distance.

Algorithm	Time (ms)	Visited vertices	Path length
Dijkstra 1	0.387	48	35
Dijkstra 2	0.509	48	35
Dijkstra 3	0.383	48	35
A* 1	0.171	22	35
A* 2	0.168	22	35
A* 3	0.182	22	35
Bellman-Ford 1	0.825	59	35
Bellman-Ford 2	0.817	59	35
Bellman-Ford 3	0.639	59	35

5.4. Summary

Table 6. Summary of the benchmark results across the three distance regimes.

Algorithm	Short t (ms)	Short V	Medium t (ms)	Medium V	Long t (ms)	Long V
Dijkstra	0.077	10	0.204	27	0.426	48
A*	0.076	6	0.142	16	0.174	22
Bellman-Ford	0.696	59	0.652	59	0.760	59

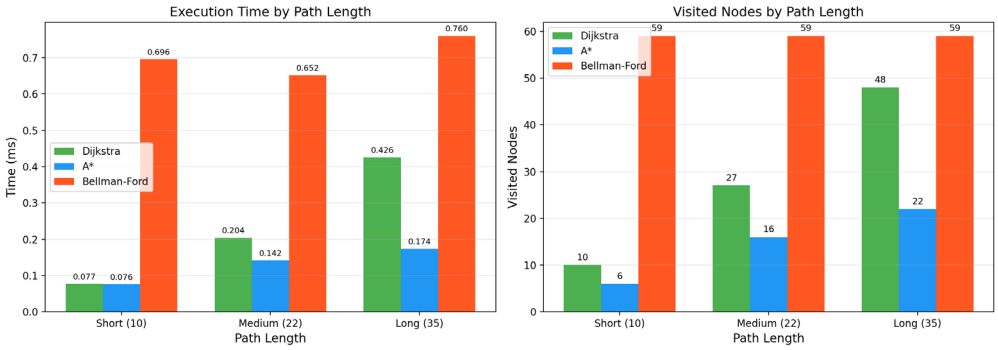

Figure 7. Side-by-side bar chart of execution time and visited vertices per algorithm.

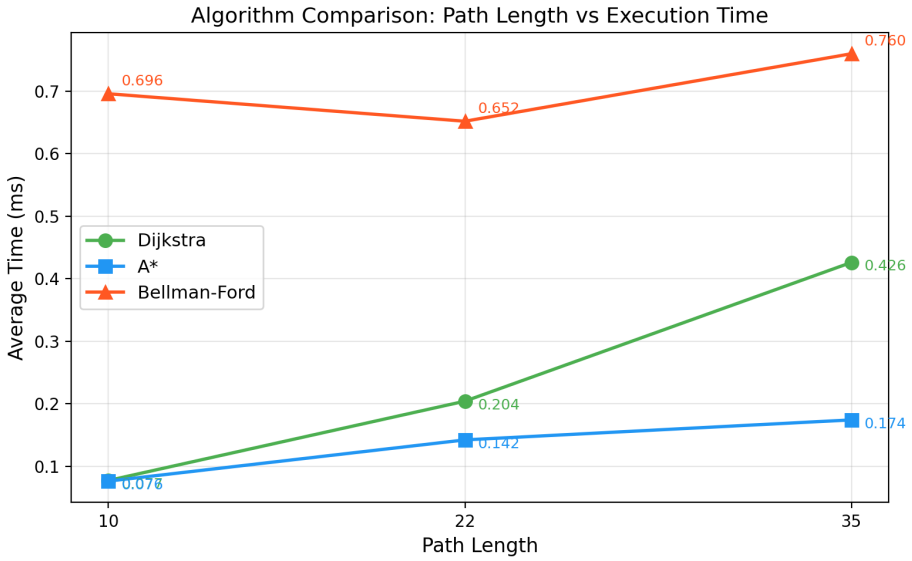

Figure 8. Line chart of average execution time as a function of path length.

6. Discussion

The experiments reveal three consistent patterns. First, A* is the fastest algorithm in all tested scenarios. At the short distance its time (0.076 ms) is essentially indistinguishable from Dijkstra (0.077 ms), but at the long distance the gap widens dramatically: A* completes in 0.174 ms while Dijkstra needs 0.426 ms, i.e. A* is more than twice as fast. This gap is explained by the octile heuristic, which directs the expansion of A* towards the target and prunes most of the search frontier that Dijkstra still expands.

Second, Dijkstra exhibits stable, predictable behaviour, but the number of vertices it visits grows roughly proportionally with the distance (10 → 27 → 48), because Dijkstra explores uniformly in all directions until the target is locked. At short distances this is harmless; on longer paths the redundancy becomes the dominant cost.

Third, Bellman-Ford is the slowest algorithm in every regime (0.652–0.760 ms), and unlike the other two its number of visited vertices is constant at 59. This is the expected consequence of relaxing every edge V − 1 times. Bellman-Ford pays this constant cost in exchange for its ability to handle negative weights, which is not a useful property in this particular game but justifies its inclusion as a reference baseline.

All three algorithms recover paths of identical length in every test, which is a strong indication that the implementations are correct. For the specific use case studied here, A* is clearly the most suitable choice; the gap with respect to Dijkstra would only grow if the game maps were enlarged.

7. Conclusions

The project successfully implements three classical shortest-path algorithms-Dijkstra, A* and Bellman-Ford-and integrates them into the game PetWars together with a step-by-step demo mode. The empirical evaluation confirms the theoretical expectations: A* is the fastest for goal-directed search on grids thanks to its admissible heuristic; Dijkstra remains optimal but explores a larger region; Bellman-Ford trades speed for generality. Possible directions for future work include the introduction of variable terrain weights (water, mountains, roads), the addition of Jump Point Search for further speed-ups on grid graphs, and the parallelisation of the algorithms for larger maps.

Funding: This research received no external funding.

Data Availability Statement: The full source code of the PetWars project and the data underlying the benchmarks in Section 5 are publicly available at *https://github.com/hristogwivanov/PetWars*.

Conflicts of Interest: The author declares no conflict of interest.

References

1. Cormen, T.H.; Leiserson, C.E.; Rivest, R.L.; Stein, C. *Introduction to Algorithms*, 3rd ed.; MIT Press: Cambridge, MA, USA, 2009.
2. Russell, S.; Norvig, P. *Artificial Intelligence: A Modern Approach*, 4th ed.; Pearson: Hoboken, NJ, USA, 2020.
3. Sapundzhi, F.; Danev, K.; Ivanova, A.; Popstoilov, M.; Georgiev, S. A Performance Comparison of Shortest Path Algorithms in Directed Graphs. *Eng. Proc.* **2025**, *100*, 31. https://doi.org/10.3390/engproc2025100031.
4. Hart, P.E.; Nilsson, N.J.; Raphael, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Trans. Syst. Sci. Cybern.* **1968**, *4*, 100–107.
5. Dijkstra, E.W. A Note on Two Problems in Connexion with Graphs. *Numer. Math.* **1959**, *1*, 269–271.
6. Pygame Documentation. Available online: https://www.pygame.org/docs/ (accessed on 12 February 2026).
7. Red Blob Games. Introduction to A*. Available online: https://www.redblobgames.com/pathfinding/a-star/ (accessed on 12 February 2026).